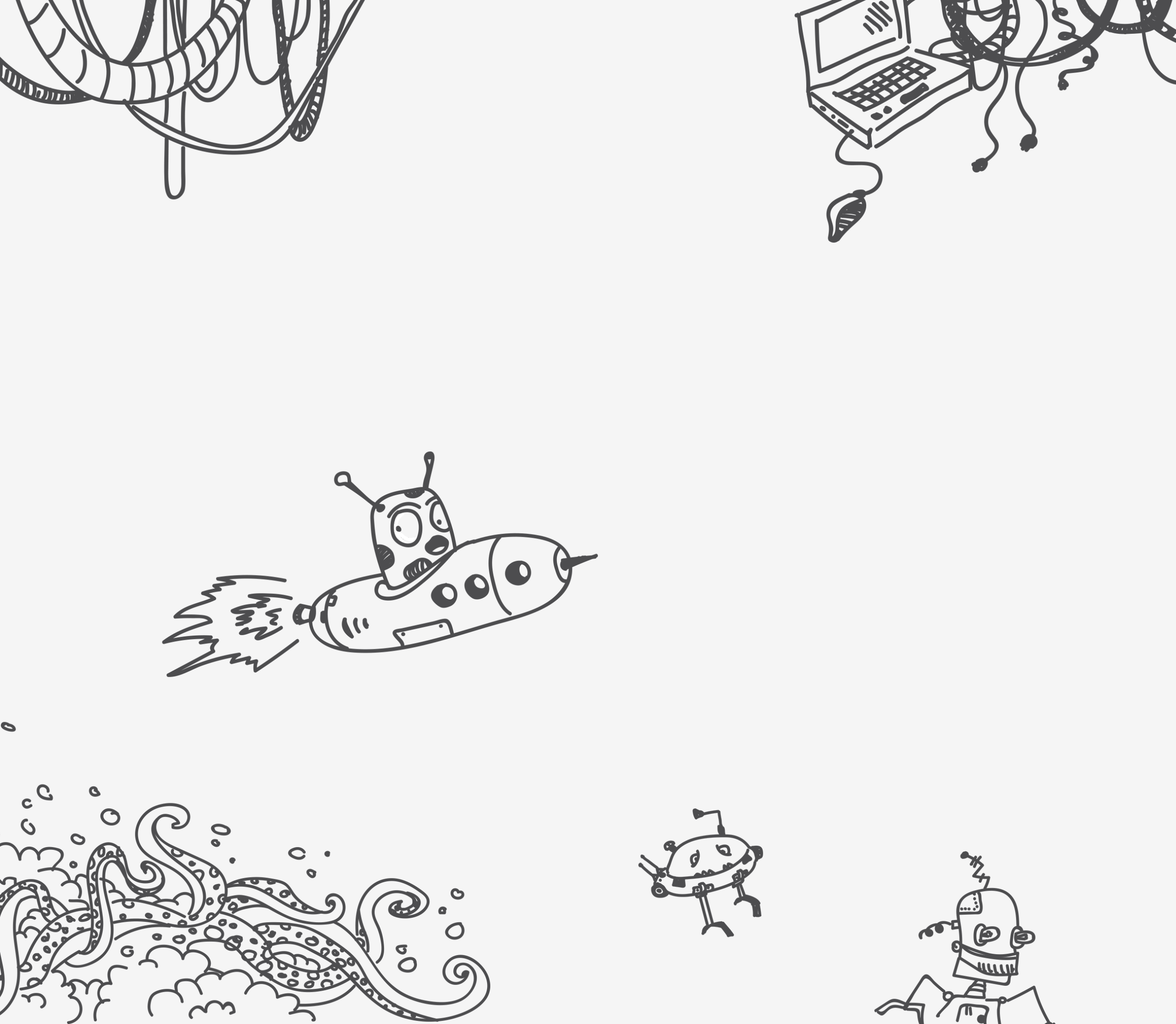


Polynomial regression - introduction and implementation

1



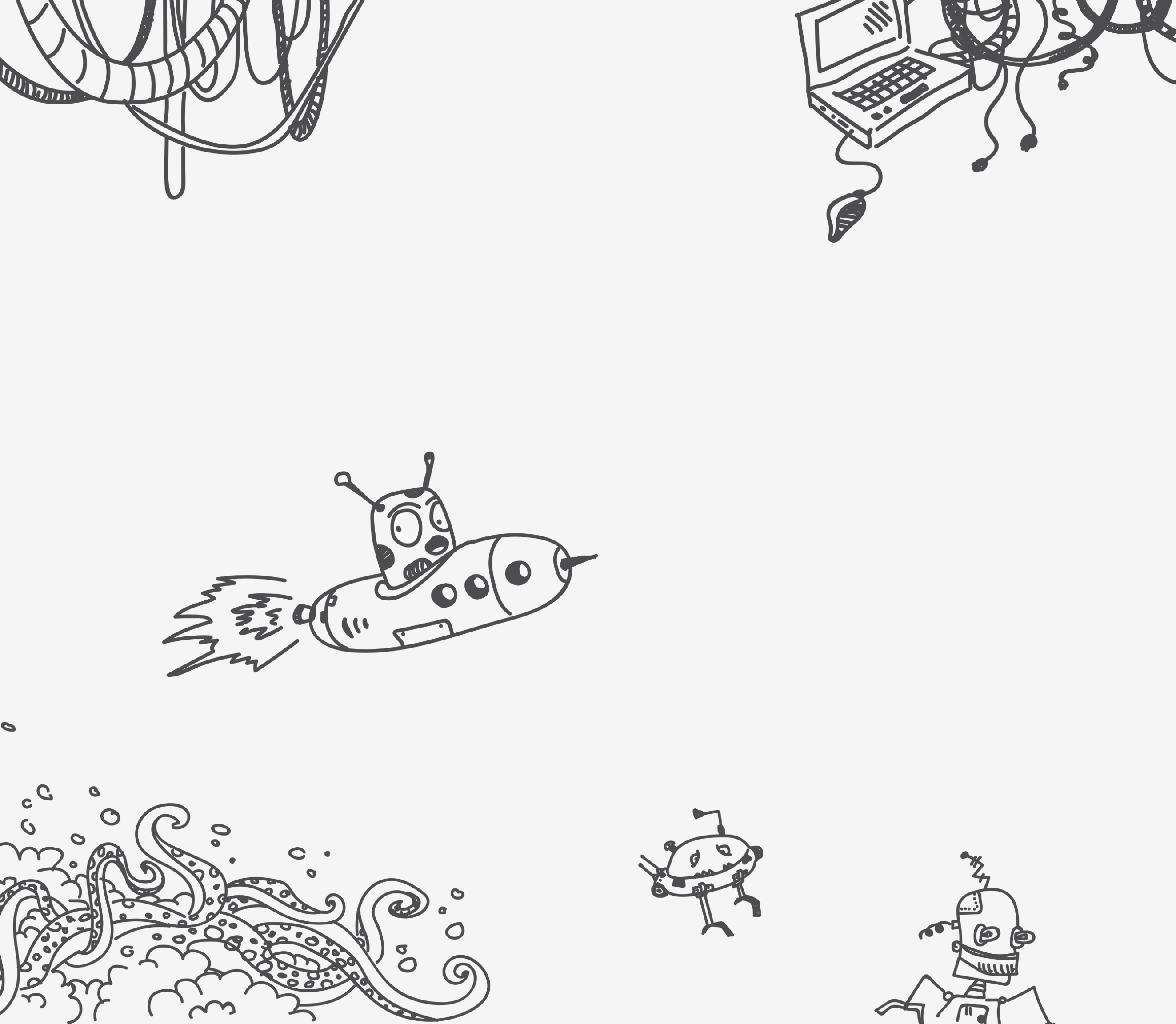
Introduction

Introduction

Supervised learning problems, such as regression, can be solved in many ways. We have already learned about the simplest linear regression model, as well as regularization techniques. For our problem, where our task is to predict the median price of real estate in particular areas of California, the linear regression model proved to be too simple.

This is because in the real world, real problems tend to be non-linear. Relationships between variables are not linear, so using a simple model is not enough.

Creating a better model is possible, for example, using polynomial regression.



Polynomial regression - definition

Polynomial regression - definition

Polynomial regression is a type of regression in which the relationships between the independent variables and the dependent variable are modeled using polynomials. Polynomials are algebraic expressions consisting, as a rule, of multiple variables.

Let's look at the formula for a simple linear regression:

$$y = ax + b + \epsilon$$

Polynomial regression - definition

In the above formula, we have two factors: ax and b . What we are interested in is the product of the directional coefficient of the regression and our variable x . The degree of the polynomial is determined by the power to which our variable is raised. In the case of linear regression, x is raised to the first power, so the expression ax is a polynomial of the first degree. If we wanted to create a polynomial of the second degree, then the form of the equation would be as follows:

$$y = a_1x + a_2x^2 + b + \epsilon$$

Our equation gained an additional expression x^2 , which was given the appropriate weight. When you look at the above formula, does it look similar to something else? The above formula is actually a popular quadratic function formula, which you may be familiar with in a slightly different form:

$$y = ax^2 + bx + c$$

Polynomial regression - definition

We know how to graphically represent a linear function - it's simple. But what about the quadratic function (polynomial of the second degree)? You may remember that a quadratic function takes the shape of a parabola, which is a more complex representation than a straight line.

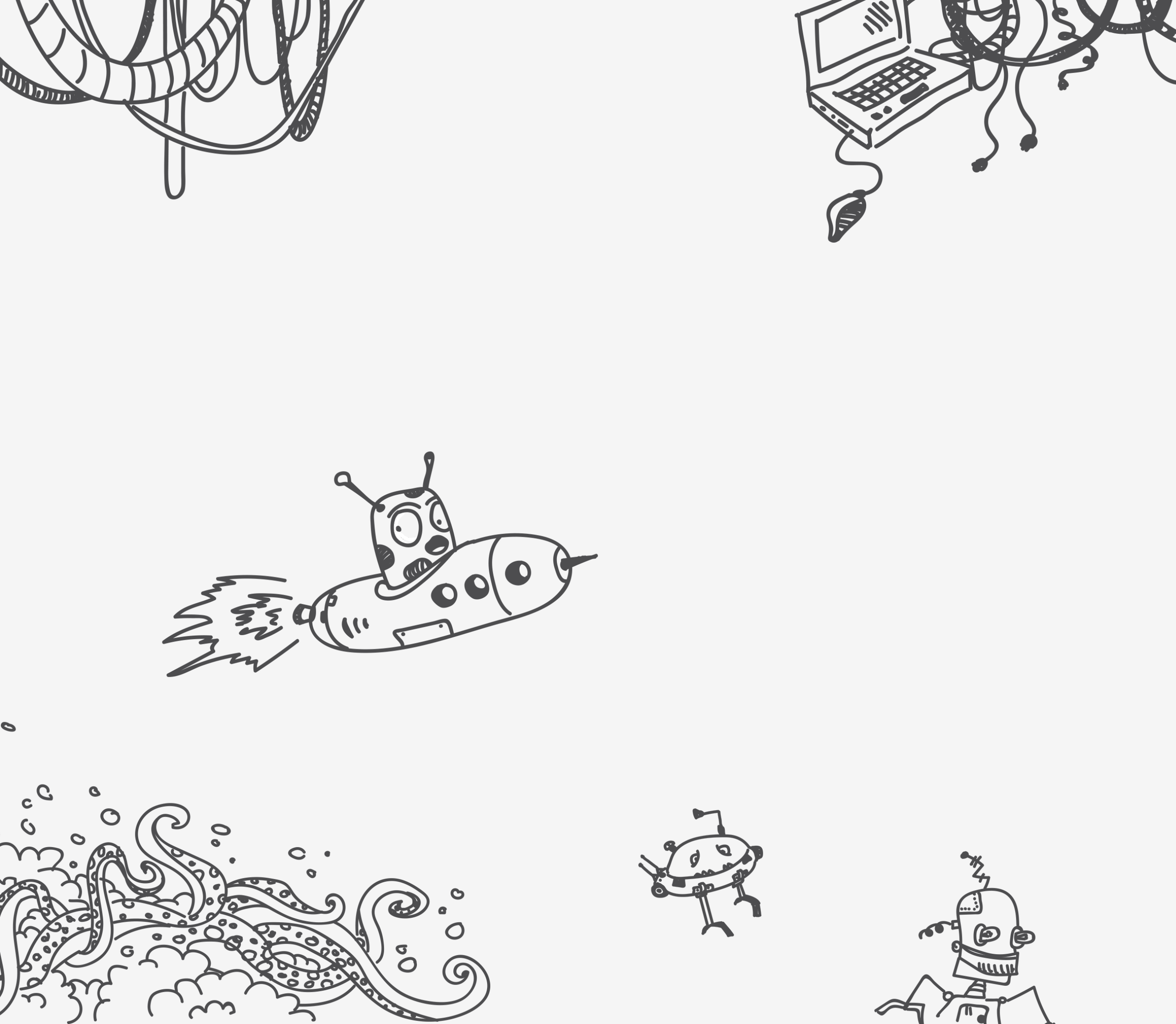
What does this mean? As the degree of the polynomial increases, its complexity increases. And this, in turn, means that the greater the degree of the polynomial, the more complex our linear model becomes (the model remains linear, because the coefficients for each variable are linear). Intuitively, you've probably guessed that when building a polynomial regression model, the degree of the polynomial should be chosen so as to avoid overtraining the model.

The general formula for polynomial regression is as follows:

$$y = a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n + b + \epsilon$$

where:

- **y** - dependent variable,
- **x** - the explanatory variable (or vector of explanatory variables),
- **b** - free expression,
- **$a_1, a_2, a_3, \dots, a_n$** - coefficients of polynomial regression,



Polynomial regression - an example in Python

Polynomial regression - an example in Python

Let's see then how it looks like to build a polynomial regression using an example. Let's generate a random dataset with one independent variable (x) and a dependent variable (y). Then we initialize the linear regression model, train it and make predictions based on the X variable. Finally, we plot our data as points and the straight line generated by the regression model:

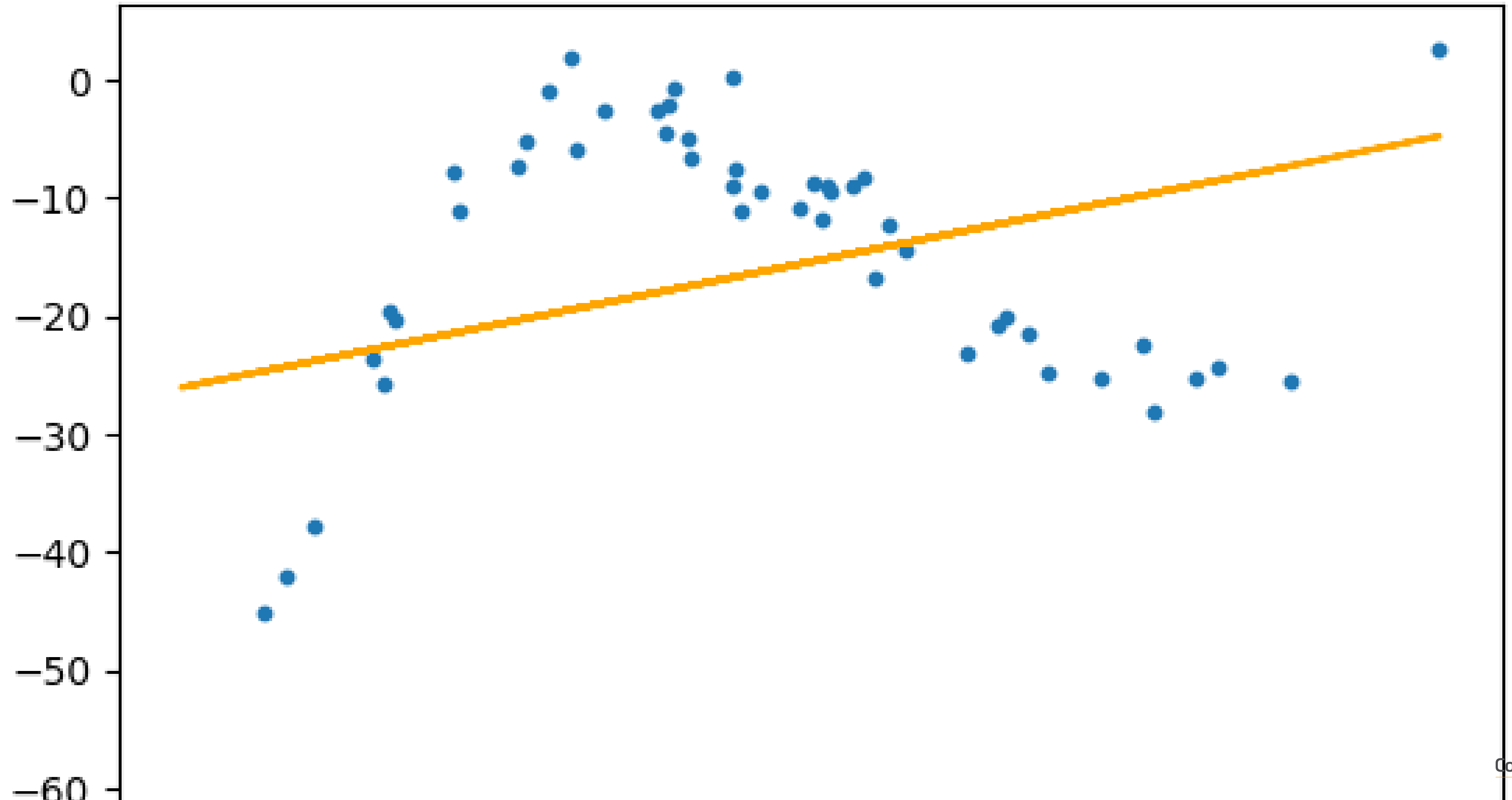
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

np.random.seed(0)
X = 2 - 3 * np.random.normal(0, 1, 50).reshape(-1, 1)
y = X - 2 * (X ** 2) + 0.2 * (X ** 3) + np.random.normal(-3, 3, 50).reshape(-1, 1)

lin_reg = LinearRegression()
lin_reg.fit(X, y)
y_pred = lin_reg.predict(X)

plt.scatter(X, y, s=10)
plt.plot(X, y_pred, color="orange")
```

Polynomial regression - an example in Python



Polynomial regression - an example in Python

In the graph, we saw how the model fitted the data. The training data was not linear and the model did not fit best. So let's count the **RMSE** and R^2 metrics for this model:

```
from sklearn.metrics import mean_squared_error, r2_score

r2 = r2_score(y, y_pred)
rmse = np.sqrt(mean_squared_error(y, y_pred))
print(f"R2: {r2}")
print(f"RMSE: {rmse}")
```

What we saw in the graph was also reflected in the metrics.

An R^2 of 0.09 indicates that the model is a very poor fit, being only slightly better than if we had used a completely random model.

Polynomial regression - an example in Python

Let's see how the situation changes when we use polynomials. To do this, we need to import the `PolynomialFeatures` class from the *preprocessing* module and create a new set of variables. To determine the degree of a polynomial, we need to pass the appropriate number to the *degree* argument:

```
from sklearn.preprocessing import PolynomialFeatures

# Create PolynomialFeatures object with degree = 2
polynomial_features = PolynomialFeatures(degree=2)

# Fit and transform on our data
X_poly = polynomial_features.fit_transform(X)
```

After creating a set of variables based on polynomials, we proceed as standard. We created the polynomial features with the **degree** parameter set to **2**, which means that we are modeling the equation $y = a_1x_1^2 + a_2x_2^2 + b$. The curve depicting the quadratic function (with a polynomial degree of 2) has the shape of a parabola. We train our linear regression model using polynomial features and make a prediction:

Polynomial regression - an example in Python

Having trained the model, let's check how accurate the model is now. We calculate the **RMSE** and R^2 as before:

```
r2 = r2_score(y, y_pred_poly)
rmse = np.sqrt(mean_squared_error(y, y_pred_poly))
print(f"R2: {r2}")
print(f"RMSE: {rmse}")
```

We see that R^2 has increased and is 0.51, which means that the model has improved significantly, but is still far from optimal. The root mean square error has also decreased, which means that the model is wrong less than a simple linear regression model.

Polynomial regression - an example in Python

Let's see in the visualization what the parabola looks like (we are dealing with a second-degree polynomial) showing the performance of the model. To generate the visualization, we need to sort the values in the arrays of **X** (our variables in their original form) and **y_pred_poly** (our prediction on the variables after the transformation):

```
import operator

plt.scatter(X, y, s=10)

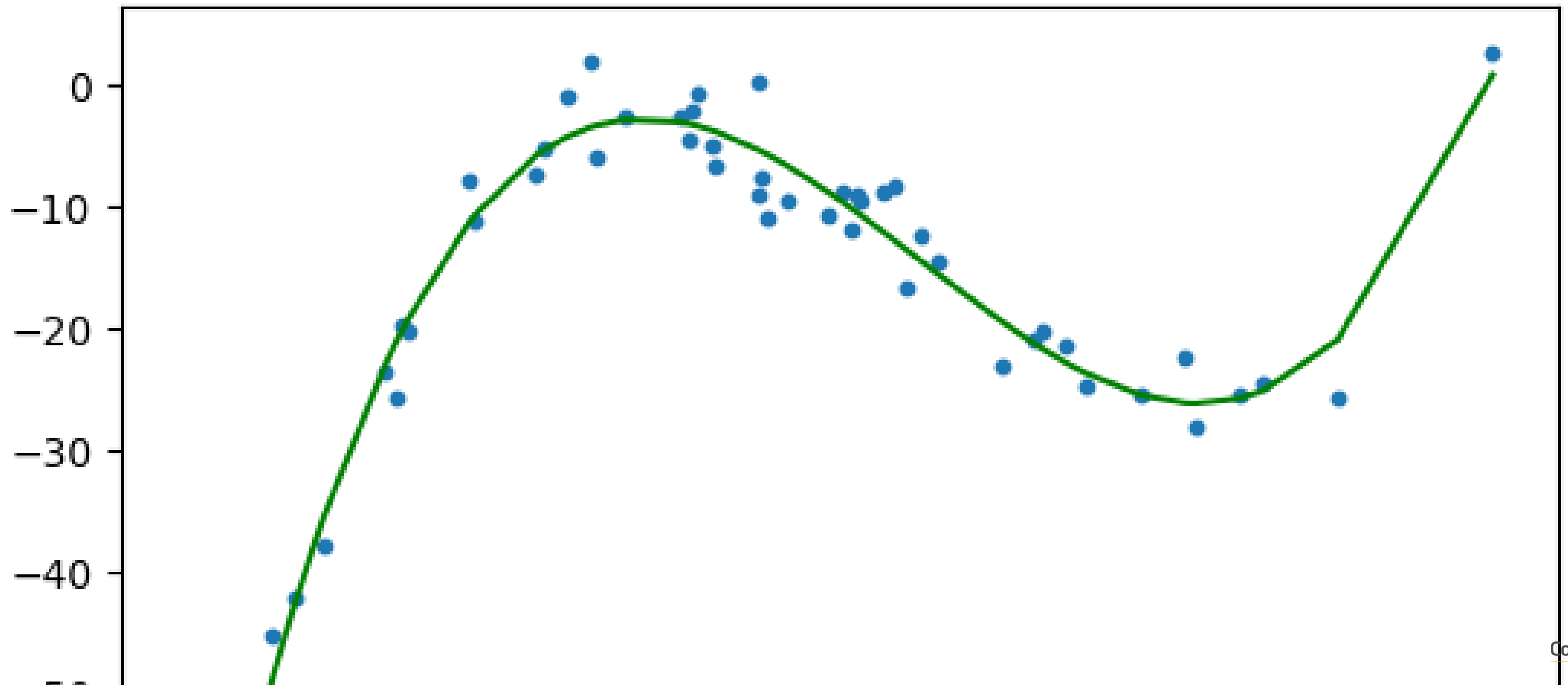
sort_axis = operator.itemgetter(0)
sorted_zip = sorted(zip(X, y_pred_poly), key=sort_axis)
X, y_pred_poly = zip(*sorted_zip)

plt.plot(X, y_pred_poly, color='green')
plt.show()
```

Let's try to build the model in the same way, just changing the degree of the polynomial to 3. This will create a slightly more complicated model, which should have better results.

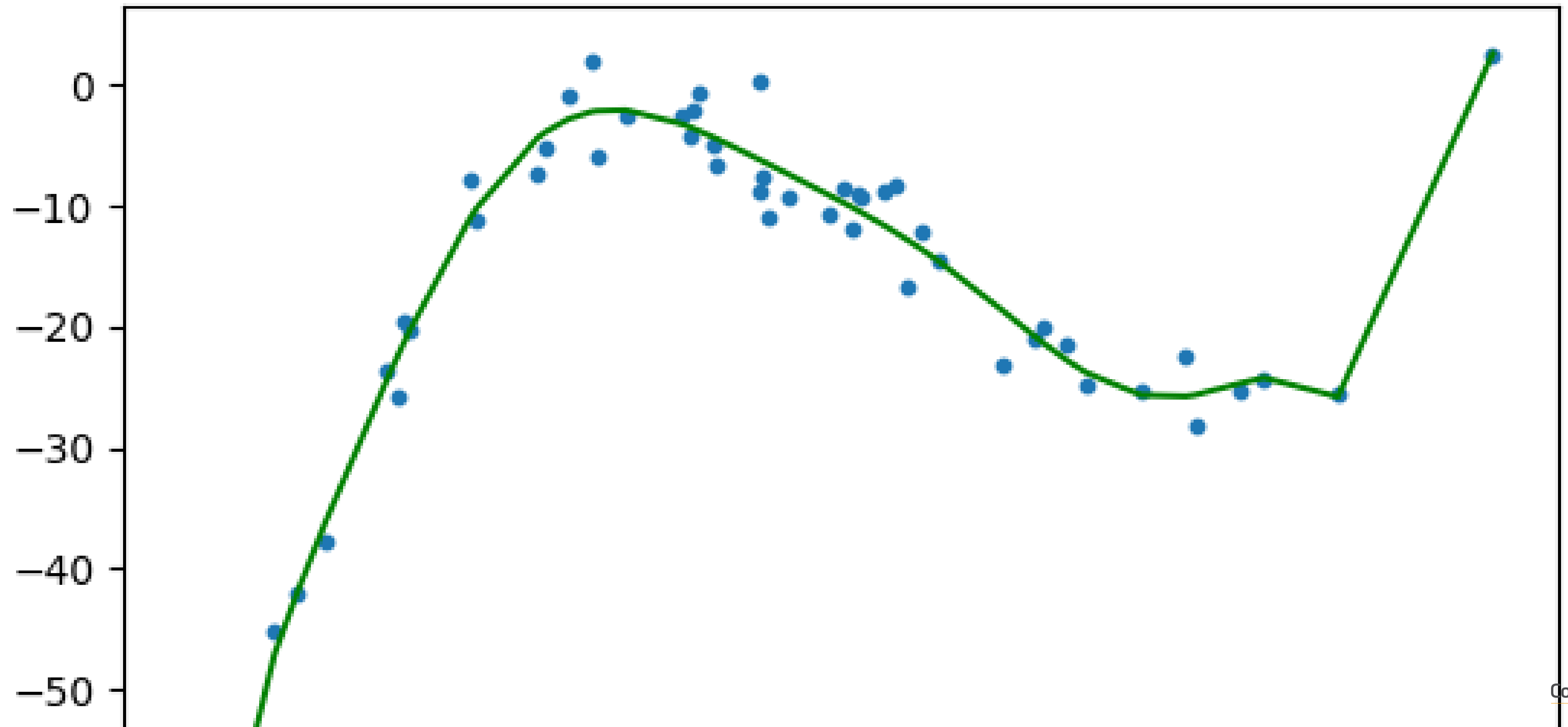
Polynomial regression - an example in Python

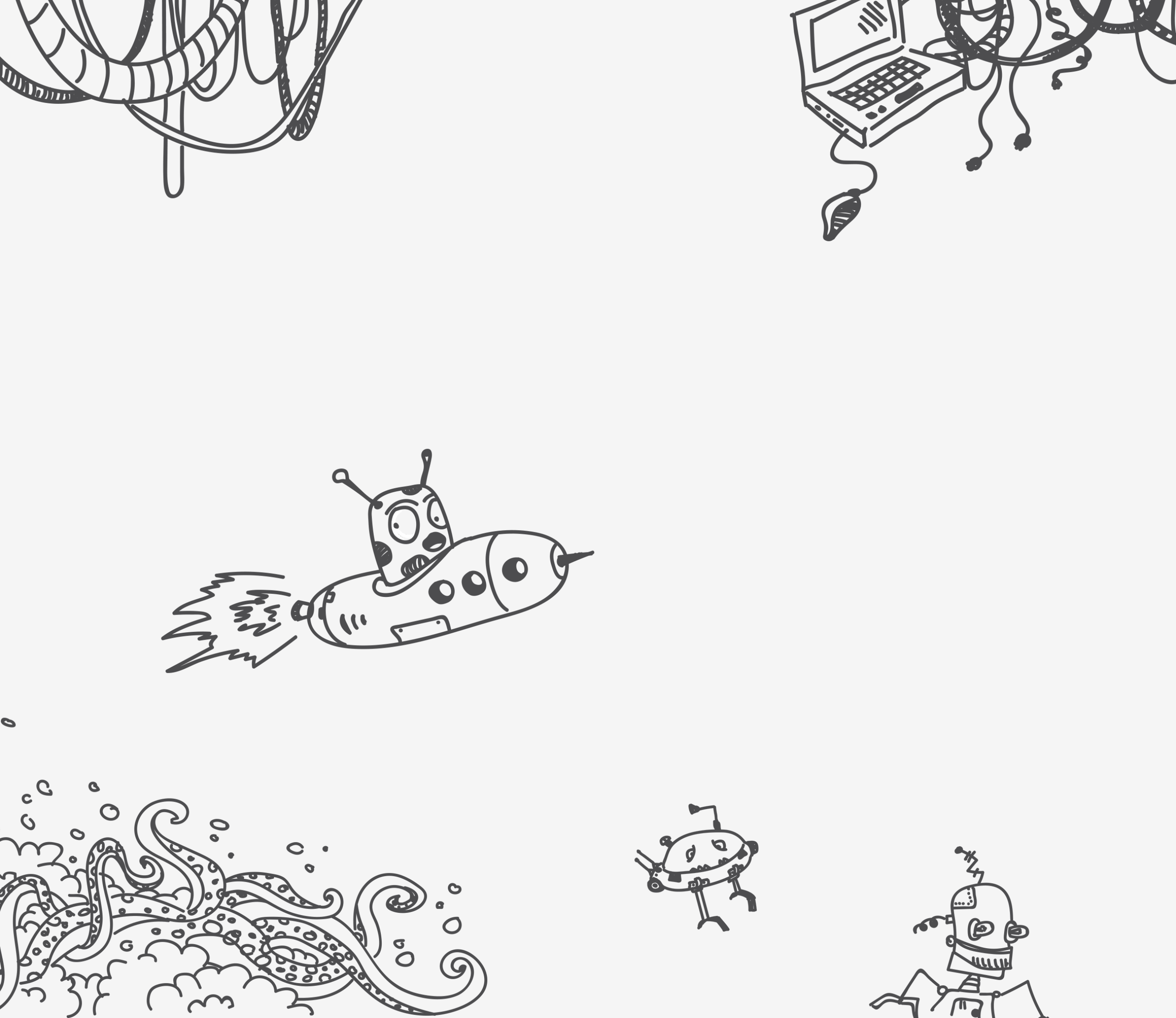
After performing the calculations, we see that the model has a coefficient of determination of 0.97, which means that it explains 97% of the relationships occurring in the data. The mean squared error has also dropped significantly and is 2.59. Let's see what the curve describing the model looks like:



Polynomial regression - an example in Python

Let's still see what the curve of a polynomial regression model with a much higher degree, such as 10, will look like:





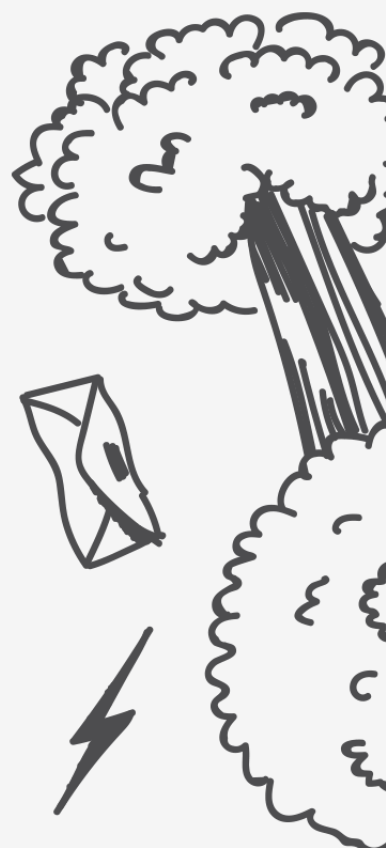
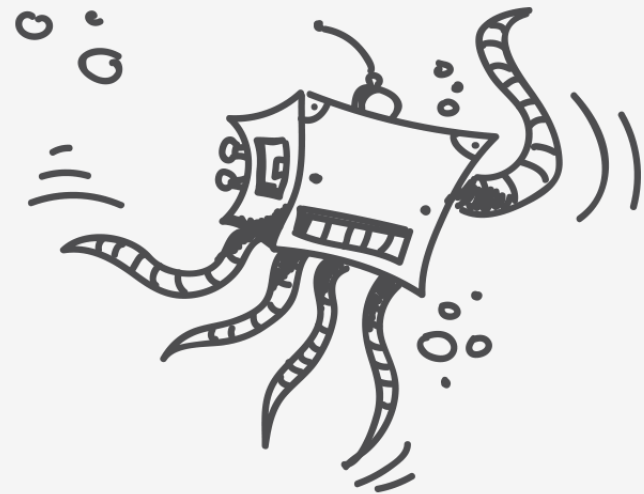
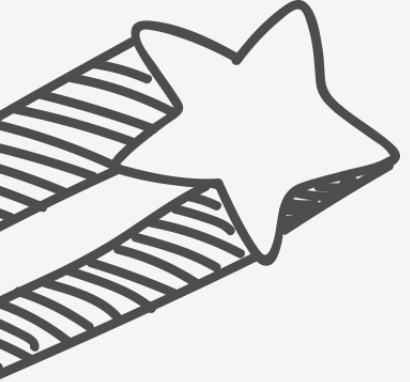
Summary

Summary

Using polynomial regression in real estate price prediction

We discussed the performance of the polynomial regression model on simple, synthetically generated data. We determined that the linear regression model (regardless of regularization) is too simple to solve more difficult problems based on data that are nonlinear. Perhaps the polynomial regression model is the solution we are looking for. We have prepared an exercise to see how such a model would perform in solving the problem we have been working with since the beginning of this day.

Good luck!



Practice time